Verifying the Stability of Terraform Executions

Eloi Perdereau[a] Daniel Sokolowski[b] Jolan Philippe[a] Hélène Coullon[c] Guido Salvaneschi[d]

CONFLANG'26
2026-10-12

[a] Université d'Orléans, France
[b] Independent Researcher, U.S.A.
[c] IMT Atlantique, Nantes, France
[d] University of St. Gallen, Switzerland

Outline

Terraform and Issues During Execution

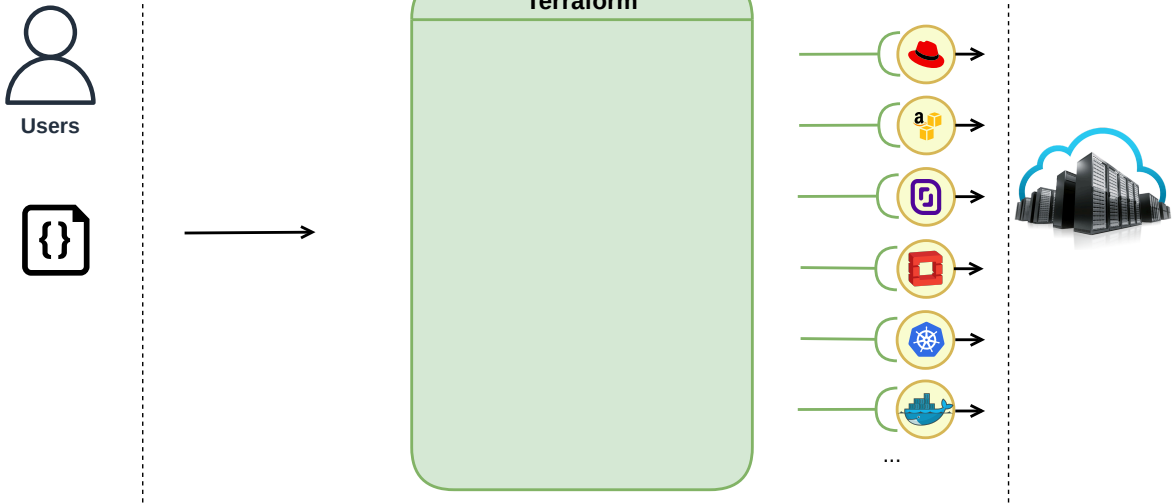
Stability Properties

Our Tool: BPlan

Experimental Validation

Terraform and Issues During Execution

Terraform Provisioning Engine



Terraform Provisioning Engine



Users



Express their *desired* state

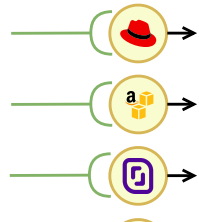
Terraform



Maintains the *current* state



Plans and **executes** changes to reconcile the states using plugin adapters



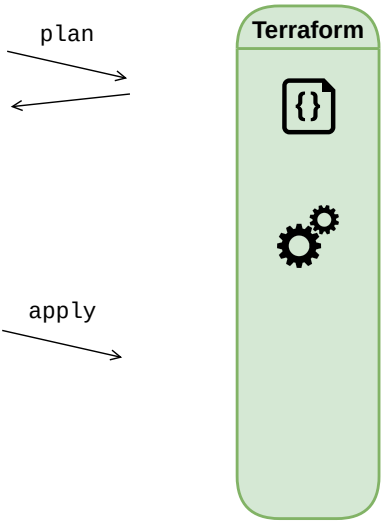
...

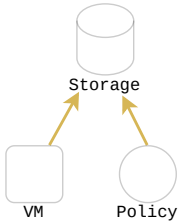


Example of an Initial Execution

```
# infra.tf

resource "g_storage_bucket" "Storage"
  name = "storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

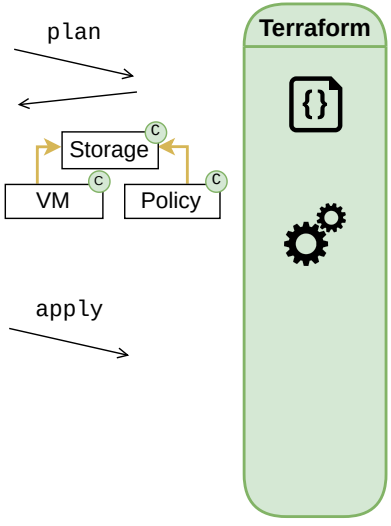


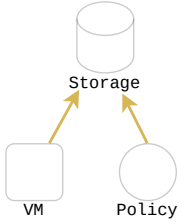


Example of an Initial Execution

```
# infra.tf

resource "g_storage_bucket" "Storage"
  name = "storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

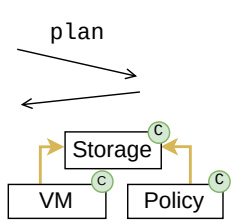


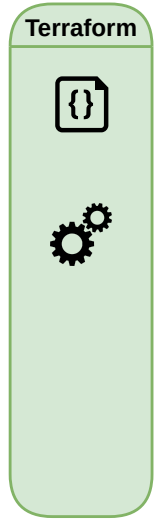


Example of an Initial Execution

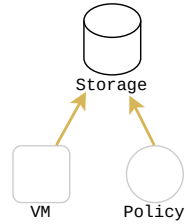
```
# infra.tf

resource "g_storage_bucket" "Storage"
  name = "storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```





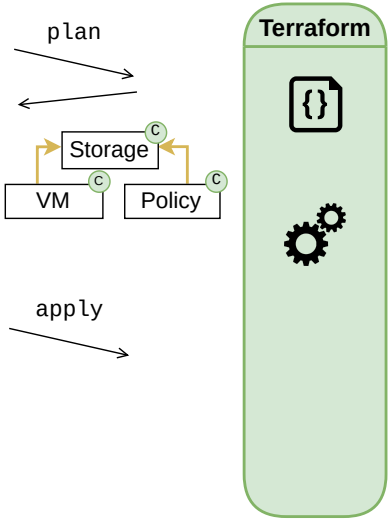
CREATE Storage

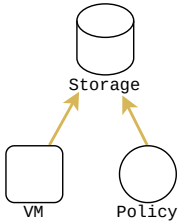


Example of an Initial Execution

```
# infra.tf

resource "g_storage_bucket" "Storage"
  name = "storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```





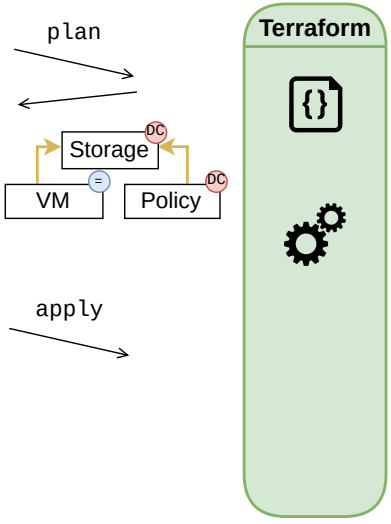
CREATE Storage

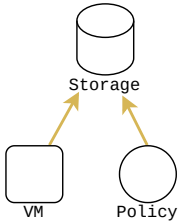
CREATE VM | CREATE Policy

Example of a Change Execution (v1: Two Delete-Create Replacement)

```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed

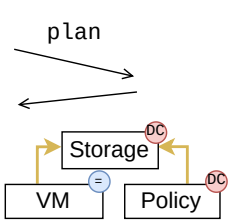


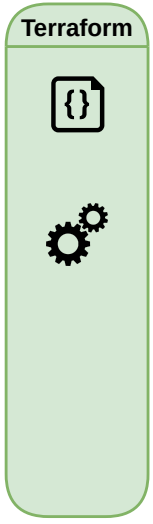


Example of a Change Execution (v1: Two Delete-Create Replacement)

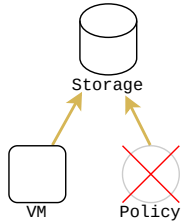
```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed





DELETE Policy



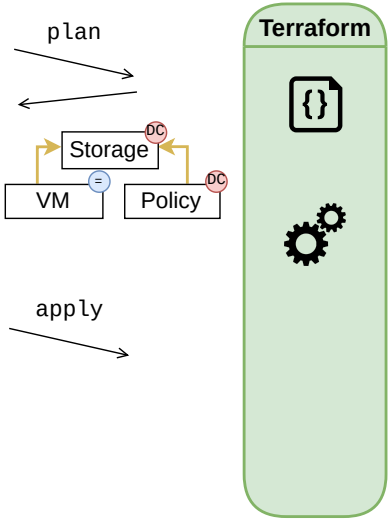
Example of a Change Execution (v1: Two Delete-Create Replacement)

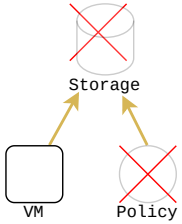
```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed

plan

apply



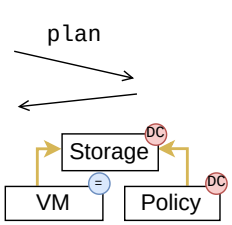


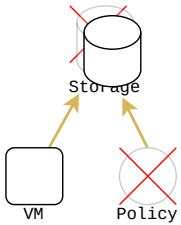

```
DELETE Policy
DELETE Storage
```

Example of a Change Execution (v1: Two Delete-Create Replacement)

```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed





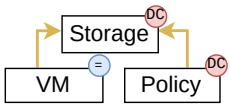

```
DELETE Policy
DELETE Storage
CREATE Storage
```

Example of a Change Execution (v1: Two Delete-Create Replacement)

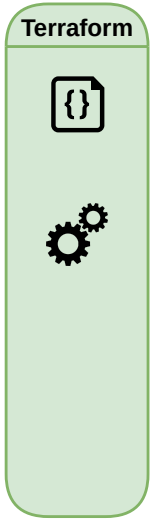
```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

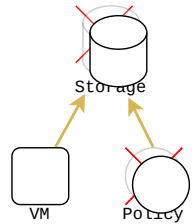
changed

plan



apply



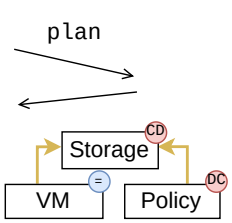


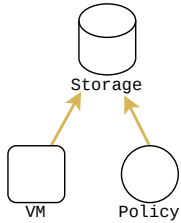

```
DELETE Policy
DELETE Storage
CREATE Storage
CREATE Policy
```

Example of a Change Execution (v2: Storage Replaced with Create-Delete)

```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = true
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed

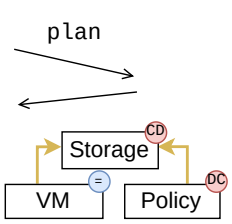


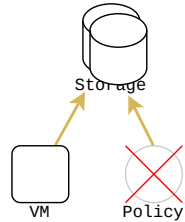


Example of a Change Execution (v2: Storage Replaced with Create-Delete)

```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = true
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed



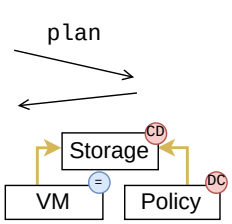


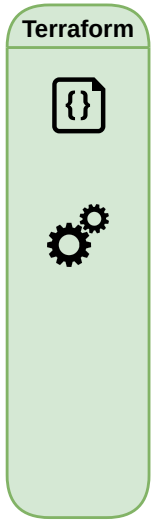
CREATE Storage	DELETE Policy

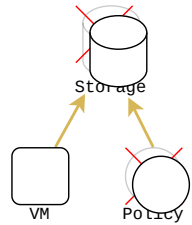
Example of a Change Execution (v2: Storage Replaced with Create-Delete)

```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = true
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed







CREATE Storage	DELETE Policy
DELETE Storage	CREATE Policy

Stability Properties

Safety Properties Based on State Predicates

Faulty predicates to characterize instability

$duplicate(r)$: resource r has two instances at the same time

$dangling(d)$: dependency d has its target instance missing

Safety Properties Based on State Predicates, Expressed in LTL[a]

Faulty predicates to characterize instability

$duplicate(r)$: resource r has two instances at the same time

$dangling(d)$: dependency d has its target instance missing

Stability properties - For P a faulty predicate

$$\text{STABLE}_P = \Box\neg P$$

$$\text{E-STABLE}_P = \Diamond P \wedge \Diamond\text{STABLE}_P \qquad \text{(eventually stable)}$$

$$\text{P-UNSTABLE}_P = \neg(\text{STABLE}_P \vee \text{E-STABLE}_P) = \Box\Diamond P \qquad \text{(perpetually unstable)}$$

[a] Linear Temporal Logic

$\Diamond\phi$ means ϕ will hold in some future (eventually)

$\Box\phi$ means ϕ holds in all futures (always)

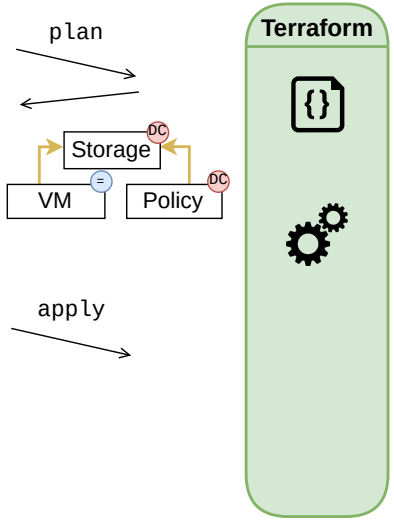
Example of Eventual Stability - Dangling Dependency

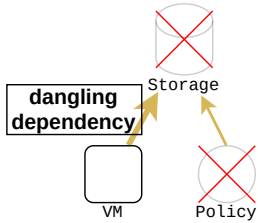
```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed

plan

apply





DELETE Policy

DELETE Storage

CREATE Storage

CREATE Policy

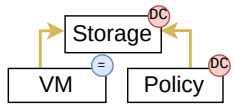
The execution is E-STABLE$_{dangling}$: there is a faulty state

Example of Eventual Stability - Dangling Dependency

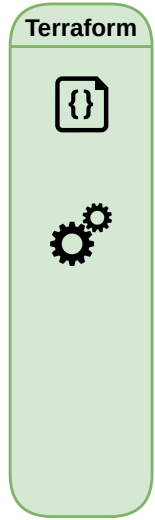
```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

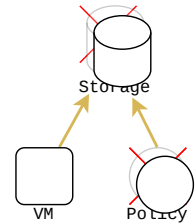
changed

plan



apply






```
DELETE Policy
DELETE Storage
CREATE Storage
CREATE Policy
```

The execution is E-STABLE$_{dangling}$: there is a faulty state, but it ends well

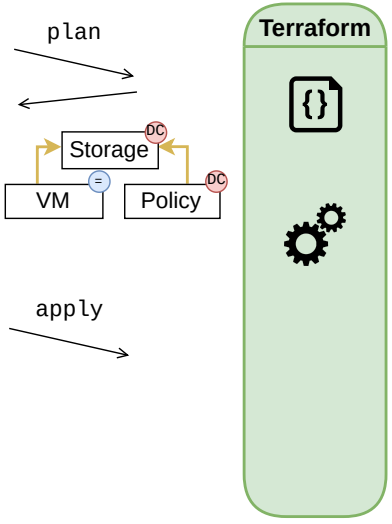
Perpetual Instability Occurs Only if Terraform Crashes

```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed

plan

apply

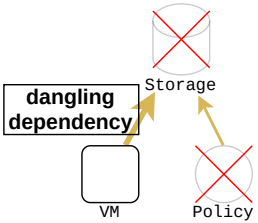


DELETE Policy

DELETE Storage

CREATE Storage

CREATE Policy



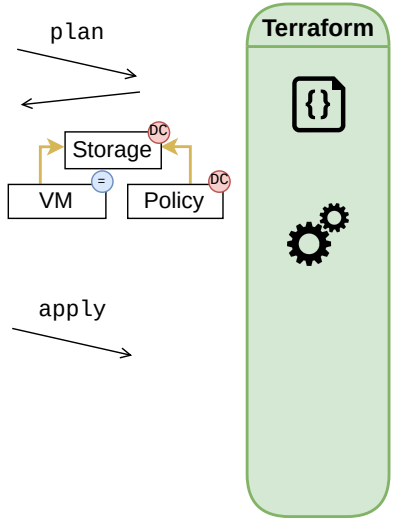
Perpetual Instability Occurs Only if Terraform Crashes

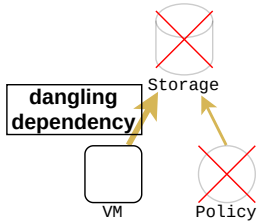
```
# infra.tf
resource "g_storage_bucket" "Storage"
  name = "new-storage-bucket"
  labels = {
    x = "my-label"
  }
  lifecycle {
    create_before_destroy = false
  }
}
resource "g_compute_instance" "VM" {
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
    }
  }
  description =
    g_storage_bucket.Storage.labels["x"]
}
resource "g_storage_bucket_iam" "Policy" {
  bucket = g_storage_bucket.Storage.name
}
```

changed

plan

apply





DELETE Policy

DELETE Storage

CREATE Storage

CREATE Policy

Execution stops at this point

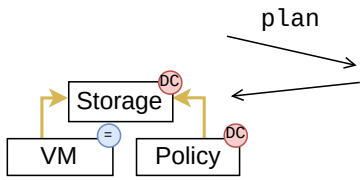
The execution is P-UNSTABLE$_{dangling}$: the execution "ends" in a faulty state

Our Tool: BPlan

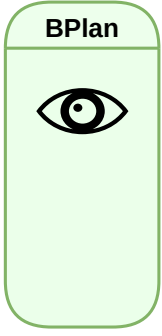
BPlan Captures Instabilities Apriori



Users







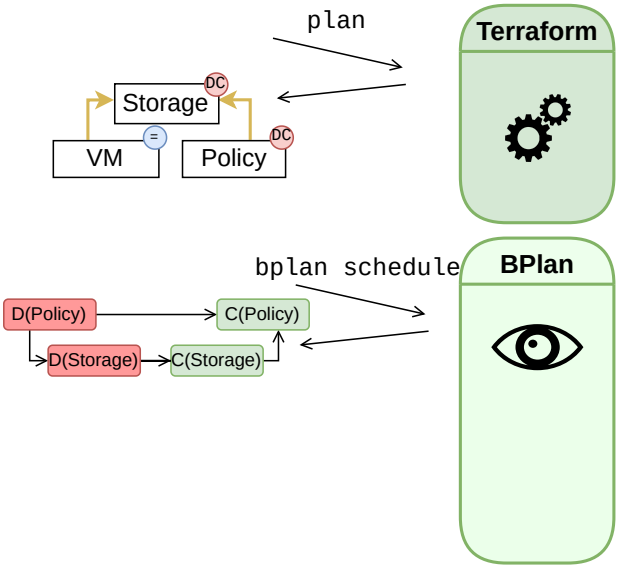
BPlan

- Reads the Terraform plan

BPlan Captures Instabilities Apriori



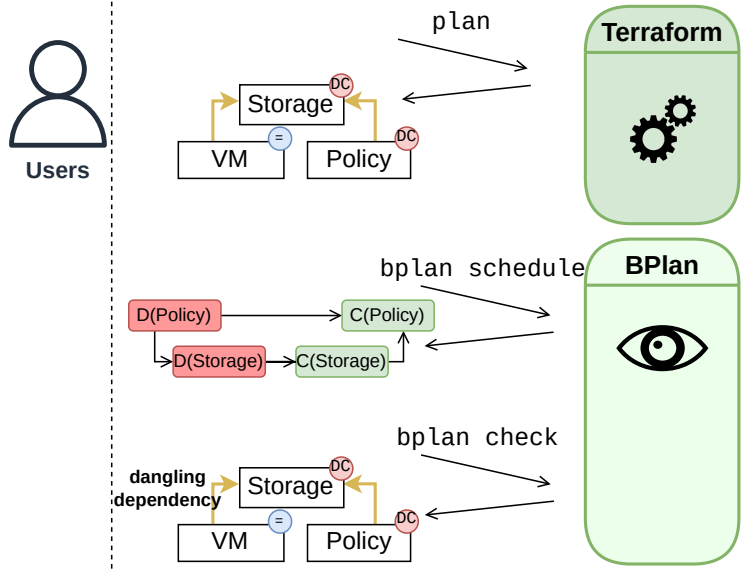
Users



BPlan

- Reads the Terraform plan
- **Computes the schedule**

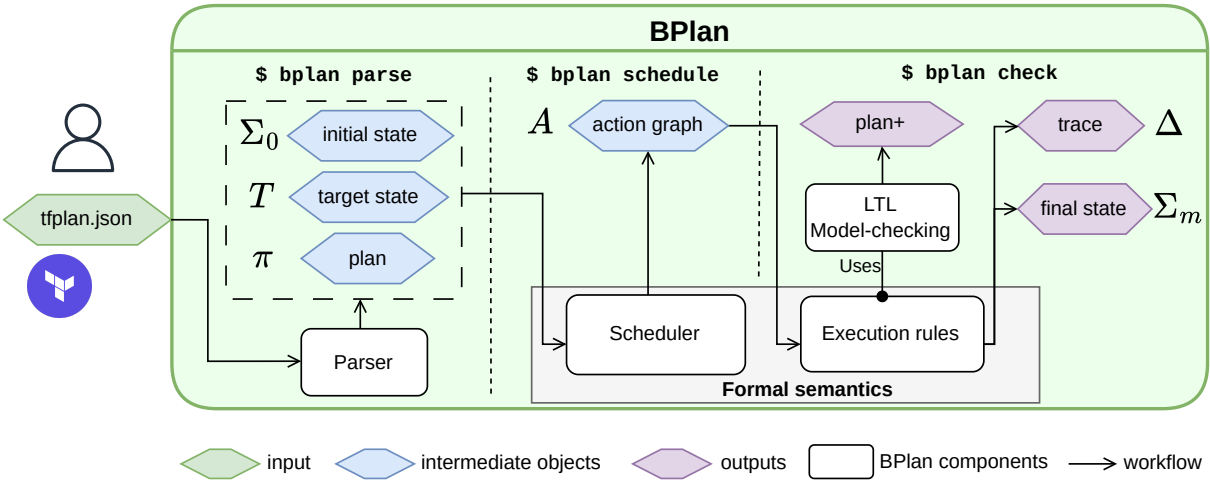
BPlan Captures Instabilities Apriori



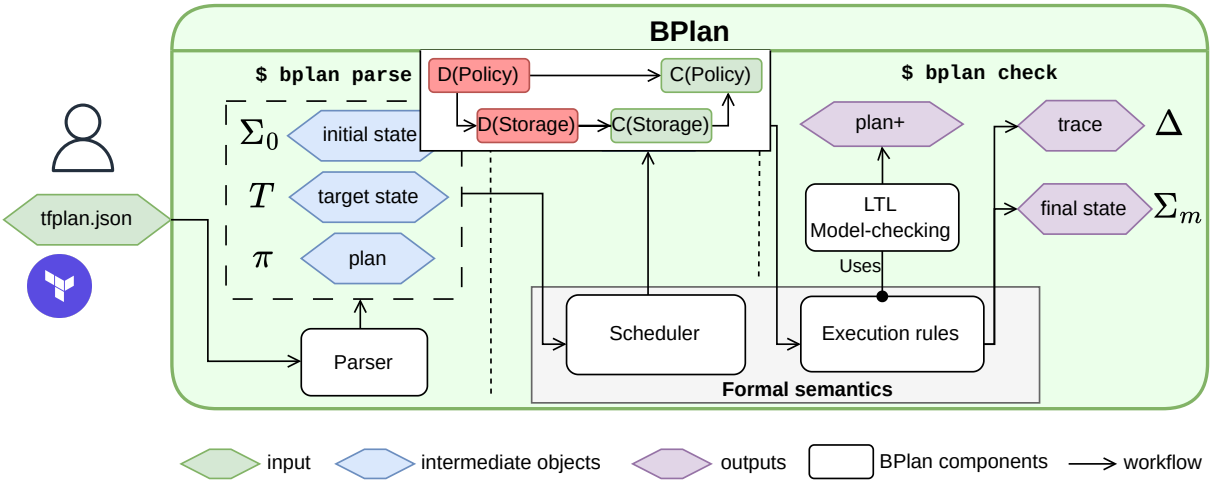
BPlan

- Reads the Terraform plan
- **Computes the schedule**
- **Verifies the properties**

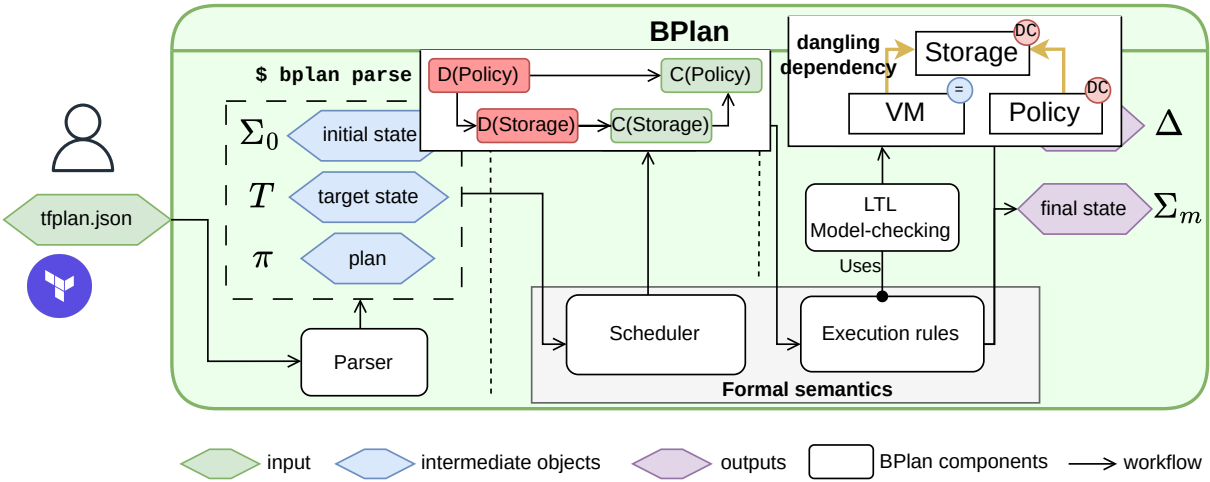
How BPlan Works



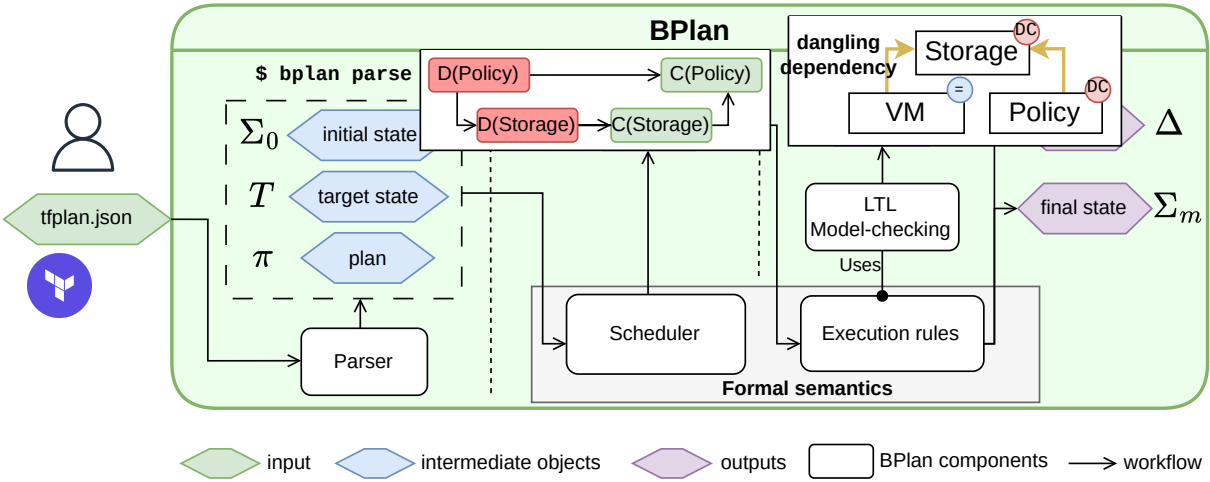
How BPlan Works



How BPlan Works



How BPlan Works



Parser limitations: (1) no group of resources and (2) one module at a time

Execution Rules

$$\frac{a(r) \in V_A \qquad \nexists(_, a(r)) \in E_A \qquad a(r) \notin \mathcal{Q}}{\Sigma, A, \mathcal{Q}, \Delta \longrightarrow \Sigma, A, \mathcal{Q} \cup \{a(r)\}, \Delta \oplus [\uparrow a(r)]}\,[StartAction]$$

$$\frac{\begin{array}{c}\mathtt{C}(r) \in \mathcal{Q} \qquad R'_\Sigma = R_\Sigma \cup \{r_{\mathtt{new}}\} \qquad D'_\Sigma = (D_\Sigma \setminus D_{\mathtt{old}}) \cup D_{\mathtt{new}} \\ D_{\mathtt{old}} = \{(r_{\mathtt{old}}, _) \in D_\Sigma\} \cup \{(_, r_{\mathtt{old}}) \in D_\Sigma\} \\ D_{\mathtt{new}} = (\{r_{\mathtt{new}}\} \times newest(D_T(r), R_\Sigma)) \cup \{(r'_\sigma, r_{\mathtt{new}}) \mid (r'_\sigma, r_{\mathtt{old}}) \in D_\Sigma\}\end{array}}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R'_\Sigma, D'_\Sigma), A \setminus \mathtt{C}(r), \mathcal{Q} \setminus \{\mathtt{C}(r)\}, \Delta \oplus [\downarrow \mathtt{C}(r)]}\,[EndCreate]$$

$$\frac{\begin{array}{c}\mathtt{U}(r) \in \mathcal{Q} \\ D'_\Sigma = (D_\Sigma \setminus (\{r_{\mathtt{old}}\} \times D_\Sigma(r_{\mathtt{old}})) \cup (\{r_{\mathtt{old}}\} \times newest(D_T(r), R_\Sigma))\end{array}}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R_\Sigma, D'_\Sigma), A \setminus \mathtt{U}(r), \mathcal{Q} \setminus \{\mathtt{U}(r)\}, \Delta \oplus [\downarrow \mathtt{U}(r)]}\,[EndUpdate]$$

$$\frac{\mathtt{D}(r) \in \mathcal{Q} \qquad R'_\Sigma = R_\Sigma \setminus \{r_{\mathtt{old}}\} \qquad D'_\Sigma = D_\Sigma \setminus (\{r_{\mathtt{old}}\} \times D_\Sigma(r))}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R'_\Sigma, D'_\Sigma), A \setminus \mathtt{D}(r), \mathcal{Q} \setminus \{\mathtt{D}(r)\}, \Delta \oplus [\downarrow \mathtt{D}(r)]}\,[EndDelete]$$

Execution Rules

$$\frac{a(r) \in V_A \qquad \nexists(_, a(r)) \in E_A \qquad a(r) \notin \mathcal{Q}}{\Sigma, A, \mathcal{Q}, \Delta \longrightarrow \Sigma, A, \mathcal{Q} \cup \{a(r)\}, \Delta \oplus [\uparrow a(r)]}\,[StartAction]$$

$$\frac{\begin{array}{c}\mathtt{C}(r) \in \mathcal{Q} \qquad R'_\Sigma = R_\Sigma \cup \{r_{\mathtt{new}}\} \qquad D'_\Sigma = (D_\Sigma \setminus D_{\mathtt{old}}) \cup D_{\mathtt{new}} \\ D_{\mathtt{old}} = \{(r_{\mathtt{old}}, _) \in D_\Sigma\} \cup \{(_, r_{\mathtt{old}}) \in D_\Sigma\} \\ D_{\mathtt{new}} = (\{r_{\mathtt{new}}\} \times newest(D_T(r), R_\Sigma)) \cup \{(r'_\sigma, r_{\mathtt{new}}) \mid (r'_\sigma, r_{\mathtt{old}}) \in D_\Sigma\}\end{array}}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R'_\Sigma, D'_\Sigma), A \setminus \mathtt{C}(r), \mathcal{Q} \setminus \{\mathtt{C}(r)\}, \Delta \oplus [\downarrow \mathtt{C}(r)]}\,[EndCreate]$$

$$\frac{\begin{array}{c}\mathtt{U}(r) \in \mathcal{Q} \\ D'_\Sigma = (D_\Sigma \setminus (\{r_{\mathtt{old}}\} \times D_\Sigma(r_{\mathtt{old}})) \cup (\{r_{\mathtt{old}}\} \times newest(D_T(r), R_\Sigma))\end{array}}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R_\Sigma, D'_\Sigma), A \setminus \mathtt{U}(r), \mathcal{Q} \setminus \{\mathtt{U}(r)\}, \Delta \oplus [\downarrow \mathtt{U}(r)]}\,[EndUpdate]$$

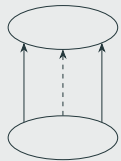
$$\frac{\mathtt{D}(r) \in \mathcal{Q} \qquad R'_\Sigma = R_\Sigma \setminus \{r_{\mathtt{old}}\} \qquad D'_\Sigma = D_\Sigma \setminus (\{r_{\mathtt{old}}\} \times D_\Sigma(r))}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R'_\Sigma, D'_\Sigma), A \setminus \mathtt{D}(r), \mathcal{Q} \setminus \{\mathtt{D}(r)\}, \Delta \oplus [\downarrow \mathtt{D}(r)]}\,[EndDelete]$$

Experimental Validation

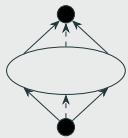
Test suite

Synthetic generation

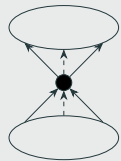
- *All possible cases*, up to 4 resources: 10,455 cases
- Randomized cases on *various topologies*, up to 50 resources: 2,176 cases



Bipartite



Diamond



Butterfly



Chain

Real modules

- Five public modules from TerraDS[1]

[1] TerraDS: A Dataset for Terraform HCL Programs - Bühler, Spielmann, Meier, Salvaneschi - MSR 2024

Correctness

Compare the traces

Run Terraform on all cases and record the *trace*

Run BPlan and

1. Check that the *schedule is compatible* with the trace
2. Check that the Terraform *trace can be produced by our semantics* exist

All cases were checked correct but

Scenarios are restricted: No addition or removal of resources

Scalability Results per Topology

Topology	Size	#T-O[a]	#STABLE	#E-STABLE	Avg STABLE	Avg E-STABLE
bipartite	≤ 25	38.3%	19.2%	42.5%	0.24s	0.08s
	> 25	25.0%	0.0%	75.0%		1.51s
diamond	≤ 25	48.0%	7.3%	44.7%	118.97s	0.71s
	> 25	50.5%	0.0%	49.5%		12.34s
butterfly	≤ 25	65.2%	18.3%	16.5%	14.92s	2.23s
	> 25	95.8%	0.0%	4.2%		133.14s
chain	≤ 25	40.0%	21.7%	38.3%	20.05s	26.16s
	> 25	84.7%	0.0%	15.3%		297.68s

[a] Timeout is set to 15 minutes (900s)

Scalability Results per Topology

Topology	Size	#T-O[a]	#STABLE	#E-STABLE	Avg STABLE	Avg E-STABLE
bipartite	≤ 25	38.3%	19.2%	42.5%	0.24s	0.08s
	> 25	25.0%	0.0%	75.0%		1.51s
diamond	≤ 25	48.0%	7.3%	44.7%	118.97s	0.71s
	> 25	50.5%	0.0%	49.5%		12.34s
butterfly	≤ 25	65.2%	18.3%	16.5%	14.92s	2.23s
	> 25	95.8%	0.0%	4.2%		133.14s
chain	≤ 25	40.0%	21.7%	38.3%	20.05s	26.16s
	> 25	84.7%	0.0%	15.3%		297.68s

STABLE cases are long to verify compared to E-STABLE cases

[a] Timeout is set to 15 minutes (900s)

Scalability Results per Topology

Topology	Size	#T-O[a]	#STABLE	#E-STABLE	Avg STABLE	Avg E-STABLE
bipartite	≤ 25	38.3%	19.2%	42.5%	0.24s	0.08s
	> 25	25.0%	0.0%	75.0%		1.51s
diamond	≤ 25	48.0%	7.3%	44.7%	118.97s	0.71s
	> 25	50.5%	0.0%	49.5%		12.34s
butterfly	≤ 25	65.2%	18.3%	16.5%	14.92s	2.23s
	> 25	95.8%	0.0%	4.2%		133.14s
chain	≤ 25	40.0%	21.7%	38.3%	20.05s	26.16s
	> 25	84.7%	0.0%	15.3%		297.68s

E-STABLE cases are checked rapidly

[a] Timeout is set to 15 minutes (900s)

Scalability Results per Topology

Topology	Size	#T-O[a]	#STABLE	#E-STABLE	Avg STABLE	Avg E-STABLE
bipartite	≤ 25	38.3%	19.2%	42.5%	0.24s	0.08s
	> 25	25.0%	0.0%	75.0%		1.51s
diamond	≤ 25	48.0%	7.3%	44.7%	118.97s	0.71s
	> 25	50.5%	0.0%	49.5%		12.34s
butterfly	≤ 25	65.2%	18.3%	16.5%	14.92s	2.23s
	> 25	95.8%	0.0%	4.2%		133.14s
chain	≤ 25	40.0%	21.7%	38.3%	20.05s	26.16s
	> 25	84.7%	0.0%	15.3%		297.68s

A topology may induce more or less instability

[a] Timeout is set to 15 minutes (900s)

Scalability Results per Topology

Topology	Size	#T-O[a]	#STABLE	#E-STABLE	Avg STABLE	Avg E-STABLE
bipartite	≤ 25	38.3%	19.2%	42.5%	0.24s	0.08s
	> 25	25.0%	0.0%	75.0%		1.51s
diamond	≤ 25	48.0%	7.3%	44.7%	118.97s	0.71s
	> 25	50.5%	0.0%	49.5%		12.34s
butterfly	≤ 25	65.2%	18.3%	16.5%	14.92s	2.23s
	> 25	95.8%	0.0%	4.2%		133.14s
chain	≤ 25	40.0%	21.7%	38.3%	20.05s	26.16s
	> 25	84.7%	0.0%	15.3%		297.68s

Bipartite and *diamond* topologies scales better than *butterfly* and *chain*

[a] Timeout is set to 15 minutes (900s)

Scalability Results on Real Modules

Two change scenarios per module: one STABLE and one E-STABLE

Size	Number of updates/replaces	STABLE	E-STABLE	Topology
7	3	13.96s	0.02s	$\sim$ diamond
8	4	7.97s	0.01s	$\sim$ bipartite
12	6	627.24s	0.02s	$\sim$ diamond + bipartite
33	7	771.91s	0.13s	$\sim$ stack of bipartites
49	4	1111.20s	0.53s	$\sim$ stack of bipartites

Related Works

Error detection in Infrastructure as Code[2]

[2] Quality assurance in infrastructure as code: Issues, approaches, and open challenges - El Hayani et. al. Journal of Systems and Software 2027

Formal methods for component-based systems[3]

[3] Component-based Distributed Software Reconfiguration: A Verification-oriented Survey - Coullon et. al. - ACM Computing Surveys 2023

Related Works

Error detection in Infrastructure as Code[2]

[2] Quality assurance in infrastructure as code: Issues, approaches, and open challenges - El Hayani et. al. Journal of Systems and Software 2027

Formal methods for component-based systems[3]

[3] Component-based Distributed Software Reconfiguration: A Verification-oriented Survey - Coullon et. al. - ACM Computing Surveys 2023

Semantics of Infrastructure as Code

- SmartFrog[4] & μPuppet[5] formlizes system configuration management tools

[4] A Formal Semantics for the SmartFrog Configuration Language - Anderson, Herry - J Netw Syst Manage 2016
[5] muPuppet: A declarative subset of the puppet configuration language - Fu, Perera, Anderson - ECOOP 2017

Related Works

Error detection in Infrastructure as Code[2]

[2] Quality assurance in infrastructure as code: Issues, approaches, and open challenges - El Hayani et. al. Journal of Systems and Software 2027

Formal methods for component-based systems[3]

[3] Component-based Distributed Software Reconfiguration: A Verification-oriented Survey - Coullon et. al. - ACM Computing Surveys 2023

Semantics of Infrastructure as Code

- SmartFrog[4] & μPuppet[5] formlizes system configuration management tools

[4] A Formal Semantics for the SmartFrog Configuration Language - Anderson, Herry - J Netw Syst Manage 2016
[5] muPuppet: A declarative subset of the puppet configuration language - Fu, Perera, Anderson - ECOOP 2017

Intra-update snipping security vulnerabilities

- Häyhä[6] statically checks for security issues in CloudFormation deployments

[6] Analyzing Infrastructure as Code to Prevent Intra-update Sniping Vulnerabilities - Lepiller, Piskac, Schäf, Santolucito - TACAS 2021

Conclusion

- Stability properties about Terraform executions
- Operational semantics of scheduling and execution
- Tool for verifying instabilities

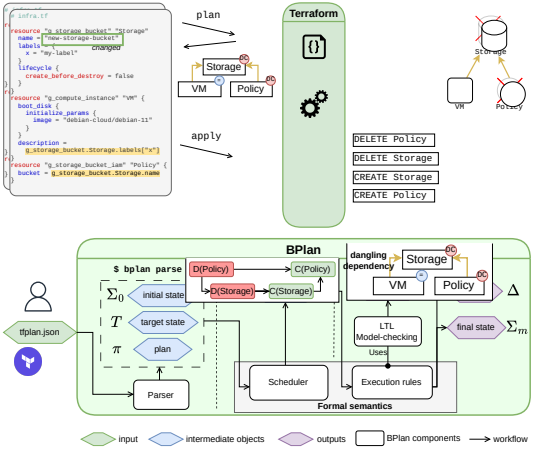
Conclusion

- Stability properties about Terraform executions
- Operational semantics of scheduling and execution
- Tool for verifying instabilities

Future work

- Augment the interpreter to overcome limitations
- Refine the semantics to better account for addition and removal of resources
- Static algorithms for verification instead of model-checking

Thank you - Questions ?



$$\frac{a(r) \in V_A \qquad \nexists(_, a(r)) \in E_A \qquad a(r) \notin \mathcal{Q}}{\Sigma, A, \mathcal{Q}, \Delta \longrightarrow \Sigma, A, \mathcal{Q} \cup \{a(r)\}, \Delta \oplus [\uparrow a(r)]}\,[StartAction]$$

$$\frac{\begin{array}{c}\mathtt{C}(r) \in \mathcal{Q} \qquad R'_\Sigma = R_\Sigma \cup \{r_{\mathtt{new}}\} \qquad D'_\Sigma = (D_\Sigma \setminus D_{\mathtt{old}}) \cup D_{\mathtt{new}} \\ D_{\mathtt{old}} = \{(r_{\mathtt{old}}, _) \in D_\Sigma\} \cup \{(_, r_{\mathtt{old}}) \in D_\Sigma\} \\ D_{\mathtt{new}} = (\{r_{\mathtt{new}}\} \times newest(D_T(r), R_\Sigma)) \cup \{(r'_\sigma, r_{\mathtt{new}}) \mid (r'_\sigma, r_{\mathtt{old}}) \in D_\Sigma\}\end{array}}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R'_\Sigma, D'_\Sigma), A \setminus \mathtt{C}(r), \mathcal{Q} \setminus \{\mathtt{C}(r)\}, \Delta \oplus [\downarrow \mathtt{C}(r)]}\,[EndCreate]$$

$$\frac{\begin{array}{c}\mathtt{U}(r) \in \mathcal{Q} \\ D'_\Sigma = (D_\Sigma \setminus (\{r_{\mathtt{old}}\} \times D_\Sigma(r_{\mathtt{old}})) \cup (\{r_{\mathtt{old}}\} \times newest(D_T(r), R_\Sigma))\end{array}}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R_\Sigma, D'_\Sigma), A \setminus \mathtt{U}(r), \mathcal{Q} \setminus \{\mathtt{U}(r)\}, \Delta \oplus [\downarrow \mathtt{U}(r)]}\,[EndUpdate]$$

$$\frac{\mathtt{D}(r) \in \mathcal{Q} \qquad R'_\Sigma = R_\Sigma \setminus \{r_{\mathtt{old}}\} \qquad D'_\Sigma = D_\Sigma \setminus (\{r_{\mathtt{old}}\} \times D_\Sigma(r))}{(R_\Sigma, D_\Sigma), A, \mathcal{Q}, \Delta \longrightarrow (R'_\Sigma, D'_\Sigma), A \setminus \mathtt{D}(r), \mathcal{Q} \setminus \{\mathtt{D}(r)\}, \Delta \oplus [\downarrow \mathtt{D}(r)]}\,[EndDelete]$$

Topology	Size	#T-O[a]	#STABLE	#E-STABLE	Avg STABLE	Avg E-STABLE
bipartite	≤ 25	38.3%	19.2%	42.5%	0.24s	0.08s
	> 25	25.0%	0.0%	75.0%		1.51s
diamond	≤ 25	48.0%	7.3%	44.7%	118.97s	0.71s
	> 25	50.5%	0.0%	49.5%		12.34s
butterfly	≤ 25	65.2%	18.3%	16.5%	14.92s	2.23s
	> 25	95.8%	0.0%	4.2%		133.14s
chain	≤ 25	40.0%	21.7%	38.3%	20.05s	26.16s
	> 25	84.7%	0.0%	15.3%		297.68s